Find the Promos

Why this problem

The brief states the bottleneck outright: an editor watches every episode by hand, and a new show ships every week. So the cost repeats forever, and unlike most things you'd point AI at, the output is checkable. A timestamp either exists in the screenplay or it doesn't. Problem 1 is also the only one of the three that arrives with real data, which meant I could show the tool working on Kuku's own material instead of on test cases I'd written to flatter it.

I scored all three against the five lenses before writing any code. Problem 1 won on quality checking and on the prototype for that same reason: a pick can be verified against its source, and the whole thing can be re-run live on an episode it has never seen. Problem 2 would have meant answering legal questions with no legal background and nothing to ground the answers in. Problem 3 was the closer call, and reading the pack did turn up real defects, including inconsistent character labels and beats that run out of order. Rather than make those the submission, I built the handling for them into the parser, which now reports 22 of them on every run.

The obvious objection is that Problem 1 is the obvious choice, so plenty of submissions will look alike. I decided to compete on method rather than topic.

Assumptions I had to make

A promo is a cut list, not a finished clip. The 20 to 60 second range and "hook inside 3 seconds" are hard limits in code, not preferences. Timestamps are relative to each episode's start, which I checked: all 50 begin at `00:00.00` and there is no series-level clock in the pack. Splicing within an episode is allowed, but only one promo crosses episodes, because stitching freely across 50 needs reliable character identity and this corpus has 64 speaker labels for 58 characters. Runtimes also don't match the "1 to 2-minute" framing: median 118.9 seconds, but 12 of the 50 run past 150 and the longest is 200.8. I went with the data.

And there is no ground truth for "good promo", so the scoring weights are mine, hand-set, written down in one table. Nothing here is learned and I won't describe it that way.

The decision everything else follows from

Code owns time. The model owns judgment. The model never sees a timestamp and never writes one. It reads beats as `[b07] JULIANA (desperate): "..."` and answers with beat IDs, then code looks each one up. A hallucinated timestamp isn't unlikely in this design, it's impossible.

That isn't a theoretical nicety. On episode 8, `qwen3.8-27b` chose a promo running `00:25.00` to `00:49.60` and named as its hook a beat starting 8.7 seconds *before* the promo begins. The description it wrote was fluent, and the moment genuinely exists in that episode. It just isn't in the clip. Nobody catches that by reading the output, because the output is internally consistent. Only checking it against the parsed beat table does, and two validators did.

How it works

Parse (code). 50 files become 3,727 timed beats. The parser reconciles labels against timestamps against beats and refuses to continue if they disagree, because a silent partial parse would produce promos from half an episode.

Score (code). This is the part I got lucky on. The screenplays already label how almost every line is delivered: `(Cold, Commanding)`, `(Screaming, Desperate)`. 1,472 of 1,487 dialogue beats carry one, across 617 values, so the tool never has to guess where the drama is. A lexicon I wrote by hand (90% hit rate) turns those tags into an intensity curve, and a promo is a shape on it: a spike in the first seconds, a rising edge, a cut while still high. From there, 18,141 legal windows on real beat boundaries, 3,376 thrown out for having nothing in the first three seconds, 171 shortlisted. 0.8 seconds, offline, free.

Select (model, one call per episode). Choose among the pre-validated windows and write the hook, cliffhanger and one line on why. It also tags a hook type from DramaBox's published taxonomy so an editor can spread types rather than posting four slaps in a row.

Validate (code). Six pass/fail checks, then near-duplicate suppression on visual descriptions. 37 dropped, ten of them real cross-episode repeats including episodes 31 and 32, which both end on a branding iron inches from the heroine's eye.

Gate (model, one call per promo). A fresh instance sees only the clip, with no logline, episode number or surrounding beats, and has to say who this is, what's at stake, what question it's left with, and whether it would keep watching.

The tool never posts anything. It turns "watch 50 episodes and scrub" into "look over 47 ranked, timestamped candidates", and the editor's keep/reject clicks get logged, which is what eventually replaces my opinion with real preferences.

How I know it's any good

Across all 92 promos, including rejected ones, `timestamps_exist` passes 92/92 and so do the other five checks. The blind gate tested 54, failed 7, left none untested, and by criterion scored ends-unresolved 1.00, character-identifiable 0.91, stakes-clear 0.76, would-keep-watching 0.65. Two runs give the identical decision fingerprint `c7b809ad92b976e7`. 42 of 50 episodes end up with a promo.

None of that answers the real question, which is whether the picks beat picking at random. So I tested it directly. Every pick goes up against a random legal window from the same episode: same length rules, same real beat boundaries, just not chosen. A blind judge sees two clips as plain text and never sees a score.

Every pair was judged twice, once with each clip shown first, because the judge turned out to prefer whichever clip it read second. Running both orders makes that cancel rather than needing a correction. The unit of analysis is the pair, not the judgement, since two judgements of the same two clips aren't independent. Against any legal window the tool was preferred in **both** orders on 24 of 38 pairs and the random window on 7, with 7 flipping: **77.4% of the 31 pairs that gave a consistent answer, p=0.003**. Against a window that already passes the hook filter: 22 to 8 with 8 flips, **73.3%, p=0.016**. Chance is 50%.

That second comparison is the more useful one. Requiring a strong opening does a lot of the work on its own, and the tension scoring plus the model's choice adds a real edge on top rather than being the whole story. Order consistency was 82% and 79%, so about a fifth of pairs were close enough that reading order decided them.

Splicing turned out to cost comprehension, which I hadn't expected to be measurable. Continuous promos pass the gate at 0.886, spliced ones at 0.800. I'd already charged a flat penalty for the cut on instinct; this is what justifies it. Across two model families agreement is only 0.50 IoU, with no overlap at all on 8 of 50 episodes and hook-type agreement of 0.48, so a real part of this ranking is model taste rather than signal. That argues for keeping the editor in the loop.

I deliberately did not build a hand-picked reference set as the headline measure. It asks whether the tool matched one person's taste, which isn't the job, and keeping one per show would recreate the manual work the tool is meant to remove.

Three times measurement killed my own idea

This is the part I'd most want to be asked about, because each one would have shipped as a plausible-looking feature that did nothing.

The duplicate check had a threshold of 0.45 on dialogue overlap. It never fired once. Measuring all 1,225 promo pairs showed dialogue overlap peaks at 0.177, because repetition in this show is visual, not verbal: the same image with different words. Switching to visual descriptions put episodes 31 and 32 at 0.191, fourth highest of 1,225, and six of the top seven pairs were one or two episodes apart.

Then the model wrote "blood and dust flying" for a beat that says "stone dust", and the word blood appears nowhere in that episode. My fix was going to be matching description words against the clip. I ran it first: it flagged 46 of 47 promos, because the frequent offenders are `declares`, `erupts` and `hanging`, which are paraphrase and structure rather than invention. Narrowed to concrete physical nouns it is 2 of 47. The real rate is about one in fifty, and the obvious verifier would have fired on everything while catching nothing.

Then I noticed that `BLOND NOBLEMAN` speaks two beats from that same hook, and blond is one character away from blood. A character label corrupted into a substance would be a far more alarming failure than embellishment, and it fits the evidence at a glance. So I tested it: same prompt, same model, speaker renamed so the string blond appears nowhere. The fabrication survived. It is cliché completion, not corruption. The more interesting explanation was the wrong one, and I would have had no evidence for it.

Scale and cost

Measured, not projected: one 50-episode show is 127 API calls and about 200,000 tokens. At a show a week that's 2,600 episodes, roughly 6,600 calls and 10.4M tokens a year. Free tier covers it outright; at Gemini Flash-Lite rates it's about $2.20 a year. Tokens are not the scaling cost and I'd say so in a planning meeting. The binding constraint on free tier is 8,000 tokens a minute, which stretches a show to roughly 20 minutes of wall clock and disappears the moment someone pays. What costs money is editor attention, which is why the gate and the duplicate suppression exist at all. Provider is a flag: Groq and Gemini are both verified end to end on different request schemas.

Where it falls over

It can't see. Shot quality, performance and pacing are invisible when your input is text, and the script tells me what gets said, not whether it lands. "Would keep watching" is the weakest gate criterion at 0.65, so clips are followable but not always compelling, and that gap is taste I can't measure. Eight episodes ship nothing, losing their best candidate to the gate or a duplicate. The alias map is still partly judgment. And the model's prose embellishes even though its timestamps cannot, which I've scoped honestly rather than patched: timestamps are verified, descriptions are not.

One more, and it's the one I'd lead with. I built reveal detection, fixed the action lexicon so that 21 unmistakably violent beats stopped scoring zero, and forced the hook into the first second, which cut dead air at the front of promos from 2.4 seconds to 0.7. It changed 8 of 43 picks and surfaced a promo I had hand-picked myself. I did not ship it, because it invalidated the A/B result and I couldn't re-measure 76 blind judgements inside the free tier's rate limit before the deadline. Shipping better promos next to a headline number describing the old ones would have been the only dishonest thing in this submission. It's in the git history.

AI tools, and where they got it wrong

Claude for code and drafting. Groq-hosted `openai/gpt-oss-120b` as the pipeline model, `qwen/qwen3.8-27b` as the comparison, Gemini to verify the provider swap. Five examples with before and after are in `docs/ai_mistakes_log.md`.

The one that mattered most was mine, not the model's. When splicing started shipping, agreement with my reference spans jumped from 0.46 to 0.519, which looked like the feature had worked. It hadn't. My IoU measured a promo's outer envelope, so a two-piece promo got credit for the seconds the splice removes. Measuring the real segment union put it back to 0.465. Nothing errored, no test failed, and I only noticed because a feature I expected to be marginal produced a 13% gain and that didn't smell right. A metric that improves when you add code deserves more suspicion than one that doesn't.